

Project 4: Preference Elicitation

Human-Centric Artificial Intelligence | TU Hamburg | SoSe 2026
Moniya Mohan (675659) and Roshin Roy (674412)

1. Task and dataset

This project compares two ways of eliciting movie preferences from a new user. In Design 1 the participant repeatedly chooses between two movies. In Design 2 the participant ranks ten movies from most to least preferred. We use the IMDB 5000 Movie Dataset, which contains 5,043 movies and metadata but no user preference labels. For a movie represented by x , the user utility is modeled as $U(x) = w^T x$. The unknown vector w represents that user's preferences and is estimated from the interactions.

2. Movie representation (Task 1)

The feature vector is kept relatively small and interpretable because w has to be estimated from only a few answers. We use movie properties that can reasonably correspond to a person's taste: genre, release period, runtime, IMDB score, popularity and content rating. High-cardinality fields such as individual actors and directors are not used.

Feature group	Representation	Count
Genres	22 multi-hot genre indicators	22
Release period	pre-1980, 1980s, 1990s, 2000s, 2010s	5
Continuous	duration, IMDB score, $\log(1 + \text{number of votes})$	3
Content rating	family, teen, mature	3
Total		33

Table 1. Feature representation used by the preference model.

Genres are multi-hot because one movie may belong to several genres. Missing years and continuous values are replaced by the median where needed. Duration, IMDB score and log vote count are standardized so their numerical scale does not dominate the utility. The remaining indicators stay as 0/1 values. Content ratings are grouped into broad family, teen and mature indicators instead of treating every rating label as a separate feature. Feature extraction and scaling are implemented in `features.py`.

3. Preference models (Task 2)

For pairwise choices we use the Bradley-Terry model. If two movies i and j have utilities U_i and U_j , the probability that the user chooses i is

$$P(i > j) = \exp(U_i) / (\exp(U_i) + \exp(U_j)).$$

For a ranking of more than two movies we use the Plackett-Luce extension. A ranking $i_1 > i_2 > \dots > i_n$ is treated as a sequence of choices: choose the best item from all n movies, then the best from the remaining $n - 1$, and continue until the order is determined.

$$P(i_1 > \dots > i_n) = \text{product}[k=1, \dots, n-1] \exp(U_{i_k}) / \text{sum}[l=k, \dots, n] \exp(U_{i_l}).$$

The model uses the same linear utility $U(x) = w^T x$ as Bradley-Terry and reduces to the pairwise case when two items remain. For both methods, w is estimated by minimizing the negative log-likelihood with an L2 penalty of $0.5 \|w\|^2$. The optimization uses L-BFGS-B. The fitting functions are implemented in `elicit.py`.

4. User study design (Task 3)

Research question and hypothesis. The study asks whether ranking a set of movies can obtain a stable preference estimate with fewer interaction screens than repeated pairwise choices. Our hypothesis is that the ranking design will reach a stable estimate sooner because one ranking contains information about the relative order of ten movies, while one pairwise screen gives only a single comparison.

Design. We use a within-subject study: every participant completes both methods. Movie preferences differ strongly between people, so using the same participant in both conditions avoids comparing two unrelated groups of tastes. The order of the pairwise and ranking blocks is randomized for each session to reduce order effects. Movies are sampled uniformly at random from the dataset, as permitted in the project specification.

Recruitment. If the study were run, we would recruit around 30 adult volunteers from TUHH student groups and mailing lists. Participants would only need to understand the interface language; they would not need to have watched all of the movies. Participation would be voluntary and responses would be analyzed without collecting identifying information.

Procedure. After reading the instructions, each participant completes both conditions in the randomized order. The implemented study contains 12 pairwise choices and three ranking tasks. Each ranking contains exactly ten movies. For every movie the interface shows title, release year, genres, IMDB score and runtime, so participants can still make a preference judgement for unfamiliar titles. Pairwise choices, complete rankings and submission timestamps are stored in the Django session.

Planned analysis. For each condition, we would fit w after each response and compare the intermediate estimate with the final estimate from that condition using cosine similarity. Before data collection we would choose a stability threshold, for example 0.90, and record how many interaction screens are needed to reach it. Because each participant completes both conditions, the two counts can be compared as paired observations. We would also check whether the randomized order changes the result. This analysis measures elicitation efficiency and stability; it does not claim that the final estimate is the user's true preference vector.

No user study was conducted for this submission, which is consistent with the project specification. The purpose here is to define the protocol and provide an interface that can collect the required preference data.

5. Interface implementation (Task 4)

The Project 4 landing page provides the two required actions: downloading this report and starting the study. Starting a session generates the random movie pools and randomly chooses which design is presented first. A short consent and instruction page is shown before the tasks.

In the pairwise block, two movie cards are shown and the participant clicks the movie they would rather watch. In the ranking block, ten movies are shown in a table and the participant assigns ranks 1 to 10. Client-side and server-side checks prevent duplicate or missing ranks. The responses are stored in the browser session, so no separate user database is needed for the prototype.

After both blocks, the application fits one preference vector from the pairwise choices with Bradley-Terry and another from the rankings with Plackett-Luce. The results page displays the strongest positive and negative fitted feature weights for each method. This final page is a check that the collected responses are connected to the implemented models; it is not presented as a user-study result because the study itself has not been run.

The implementation therefore covers the required flow: landing page and report download, randomized study start, pairwise preference elicitation, ranking of sets of ten movies, and estimation of the corresponding user preference vectors.